Datos del concurso

Planificación de cátedra

ASIGNATURA: Programación IV

CARRERA: Licenciatura en Sistemas

CARGA HORARIA SEMANAL: 8hs (128 hs totales – 1.er cuatrimestre)

PLAN DE ESTUDIO: 2025 (Res. “C.S.” 274/24)

AÑO: 2026.

RÉGIMEN: Teórico / Práctico

ASPIRANTE: Ignacio Luis Novello Pelayo

1.- INTRODUCCIÓN

Programación IV es una asignatura del tercer año de la **Carrera Licenciatura en Sistemas (Plan 2025)** de la **Facultad de Ciencias de la Administración – UNER**, con una carga horaria de 8 horas semanales en régimen teórico-práctico. Se orienta al estudio y desarrollo de aplicaciones web modernas, programación asíncrona, concurrencia y sistemas distribuidos, integrando conceptos fundamentales de la ingeniería de software con tecnologías y arquitecturas actuales de desarrollo de sistemas.

La asignatura aborda el diseño e implementación de sistemas cliente-servidor, servicios web y aplicaciones distribuidas, incorporando además fundamentos de paralelismo, comunicación entre procesos y diseño de interfaces de usuario. Estos contenidos responden a la creciente relevancia de los sistemas web y distribuidos en los ámbitos productivo, organizacional y gubernamental, constituyendo un pilar en la formación del profesional en informática.

De acuerdo con el Plan de Estudios 2025, **Programación IV** se integra al **Área Algoritmos y Lenguajes (AyL)**, la cual comprende los principios, paradigmas y metodologías de programación y el soporte brindado por distintos lenguajes para su implementación. Esta área incluye las asignaturas Algoritmos, Programación I a V y Lenguajes de Programación, constituyendo el eje central de la formación en desarrollo de software de la carrera.

Asimismo, por su contenido vinculado al desarrollo de sistemas distribuidos y aplicaciones web, la asignatura articula transversalmente con las áreas Arquitectura, Sistemas Operativos y Redes e Ingeniería de Software, Bases de Datos y Sistemas de Información, aportando a la integración de conocimientos necesarios para la construcción de sistemas informáticos complejos.

La asignatura tiene como propósito general que los estudiantes adquieran competencias para diseñar, desarrollar y desplegar aplicaciones web y sistemas distribuidos, comprendiendo los principios conceptuales y tecnológicos que sustentan estas soluciones.

Por su ubicación en el plan de estudios de la Carrera, Programación IV, constituye un punto de articulación entre la formación básica en programación y las asignaturas avanzadas de desarrollo de software.

Relación con asignaturas:

- Continúa y profundiza los contenidos de **Programación I, II y III** (paradigmas, estructuras de datos y programación orientada a objetos).
- Se apoya en **Sistemas Operativos** y **Redes de Datos** para comprender concurrencia, comunicación y arquitecturas distribuidas.
- Retoma conceptos de **Ingeniería de Software I** sobre procesos y diseño de sistemas. Aplica, además, los contenidos de Interacción Humano-Computadora al diseño concreto de interfaces web. En el segundo cuatrimestre de 2026, el trabajo final integrador dio continuidad al análisis y diseño de un sistema de gestión de incidencias y tareas de reparación para la Dirección de Informática de la Municipalidad de Concordia, desarrollado como caso de estudio en esa asignatura.
- Toma conceptos básicos de **Base de Datos I**, asignatura que se desarrolla en paralelo.
- Proporciona bases para **Programación V** (arquitecturas de aplicaciones y frameworks). Por ello, en esta asignatura se privilegia la comprensión de los fundamentos por sobre el uso intensivo de frameworks.
- Se articula con **Ingeniería de Software II** en la construcción y gestión de sistemas complejos.

- Contribuye a **Bases de Datos II**, **Seguridad en Sistemas de Información** y **Proyecto Final Integrador**, donde se aplican arquitecturas distribuidas y servicios web.

En este sentido, **Programación IV** cumple un rol integrador dentro del trayecto de formación en desarrollo de software del plan, permitiendo al estudiante pasar de la programación individual de componentes a la construcción de sistemas completos, distribuidos y escalables.

2 .- FUNDAMENTACIÓN

En el contexto del plan de estudios, **Programación IV** actúa como instancia de integración entre los conocimientos de programación adquiridos en los primeros años y las asignaturas avanzadas de arquitectura de aplicaciones, ingeniería de software y sistemas distribuidos, aportando al desarrollo progresivo de competencias profesionales vinculadas al diseño e implementación de sistemas informáticos complejos.

El eje articulador de la propuesta consiste en concebir a la Web como el sistema distribuido por excelencia. Desde la primera clase, cliente y servidor se presentan como nodos que intercambian mensajes a través de una red; el modelo asincrónico y orientado a eventos de JavaScript introduce de manera natural la problemática de la concurrencia; y la necesidad de que una aplicación web atienda a muchos usuarios en forma simultánea, confiable y escalable conduce a la programación paralela y a los algoritmos y arquitecturas distribuidas. De este modo, los contenidos mínimos no se abordan como bloques aislados, sino como etapas sucesivas y de complejidad creciente sobre un mismo objeto de estudio: una aplicación web que los estudiantes construyen y hacen evolucionar a lo largo del cuatrimestre.

Se adopta JavaScript, tanto en el navegador como en el entorno Node.js, como lenguaje principal de la asignatura. Esta decisión permite utilizar un único lenguaje en todas las capas de la aplicación y concentrar el esfuerzo en los conceptos. Para los contenidos de concurrencia y paralelismo se incorporan ejemplos en otros lenguajes (por ejemplo, Python o Java), con el fin de contrastar distintos modelos de concurrencia y evaluar su adecuación a cada contexto, en línea con la competencia CE.AyL.4.

Relación vertical (trayecto formativo):

Programación IV se apoya en los conocimientos previos de **Programación I, II y III** (paradigmas, estructuras de datos y programación orientada a objetos), Sistemas Operativos (concurrencia y procesos) y Redes de Datos (comunicación y arquitecturas de red). A partir de esta base, introduce el desarrollo web full stack, la programación asíncrona y los fundamentos de sistemas distribuidos.
A su vez, constituye un prerrequisito conceptual para asignaturas posteriores como Programación V (arquitecturas y frameworks), Ingeniería de Software II (construcción y gestión de sistemas complejos), Bases de Datos II (persistencia en sistemas distribuidos) y Proyecto Final Integrador, donde se espera que el estudiante aplique arquitecturas web y distribuidas en problemas reales.

Relación horizontal (articulación en el mismo nivel):

En el tercer año la asignatura se articula especialmente con Bases de Datos I y Redes de Datos. Con la primera, en la construcción de aplicaciones con persistencia de datos; con la segunda, en el soporte de comunicación entre componentes distribuidos y servicios web. Esta articulación permite abordar el desarrollo de sistemas completos que integran interfaz, lógica y datos en entornos de red. La persistencia de datos se aborda en Programación IV en un nivel introductorio, orientado a su uso desde la aplicación, para luego profundizarse en Bases de Datos I; y el protocolo HTTP y la arquitectura cliente-servidor se coordinan con Redes de Datos para trabajarlos de forma complementaria.

Objetivos de la asignatura

- Desarrollar aplicaciones web integrales basadas en arquitecturas cliente-servidor.
- Diseñar interfaces interactivas mediante programación del lado del cliente, aplicando criterios de usabilidad y accesibilidad.
- Implementar servicios web y APIs seguras del lado del servidor y verificar su funcionamiento mediante pruebas.
- Aplicar modelos de concurrencia y paralelismo en aplicaciones informáticas y medir su rendimiento.
- Comprender los fundamentos de los sistemas distribuidos y su implementación práctica, aplicando algoritmos de coordinación, replicación y tolerancia a fallos.

Estos objetivos se corresponden con los resultados de aprendizaje del programa, orientados a la construcción de aplicaciones web, servicios y sistemas distribuidos eficientes y escalables.

Enfoque metodológico y organización teoría-práctica

La asignatura adopta un enfoque basado en competencias y centrado en el desarrollo de proyectos, promoviendo el aprendizaje activo mediante la construcción progresiva de aplicaciones web y distribuidas. Se prioriza la resolución de problemas reales y el uso de tecnologías actuales del ecosistema de desarrollo web.

La organización didáctica integra teoría y práctica de manera articulada:

- Clases teóricas: introducción de conceptos fundamentales de la web, protocolos, arquitecturas cliente-servidor, concurrencia y sistemas distribuidos.
- Prácticas de laboratorio: desarrollo incremental de aplicaciones full stack, APIs y servicios distribuidos.
- Trabajos integradores: implementación de sistemas web completos que integran interfaz, lógica de negocio y persistencia.

Principales herramientas tecnológicas

Las prácticas se desarrollarán con tecnologías actuales de desarrollo web y distribuido, tales como:

- Lenguajes y estándares web (HTML, CSS, JavaScript).
- Frameworks y entornos de desarrollo del lado del cliente.
- Entornos de desarrollo del lado del servidor y APIs REST.
- Sistemas de control de versiones.
- Herramientas de ejecución concurrente y comunicación asincrónica.
- Contenedores y plataformas de despliegue. (como soporte para ejecutar varios nodos en las prácticas de sistemas distribuidos)

Relación con Actividades Reservadas (AR) del título

La asignatura contribuye a las siguientes Actividades Reservadas del Licenciado/a en Sistemas:

- AR1: Especificar, proyectar y desarrollar sistemas de información, sistemas de comunicación de datos y software cuya utilización pueda afectar seguridad, salud, bienes o derechos.
- AR2: Proyectar y dirigir lo referido a seguridad informática.
- AR3: Establecer métricas y normas de calidad de software.

- AR4: Certificar el funcionamiento, condición de uso o estado de sistemas informáticos.
- AR5: Dirigir y controlar la implementación, operación y mantenimiento de sistemas informáticos.

Estas actividades se vinculan con el desarrollo de aplicaciones web y distribuidas seguras y confiables abordado en la asignatura.

Relación con Alcances del título

Programación IV se relaciona especialmente con los siguientes Alcances del Licenciado/a en Sistemas:

- AL1: Planificar, dirigir y desarrollar sistemas de software y sistemas de información.
- AL3: Dirigir el análisis y diseño de sistemas de información.
- AL5: Establecer métricas y normas de calidad y seguridad de software.
- AL6: Planificar y desarrollar componentes de seguridad en sistemas de software.
- AL9: Analizar y evaluar proyectos de diseño e implementación de sistemas.
- AL10: Analizar y evaluar proyectos de redes y sistemas distribuidos.

La asignatura aporta a estos alcances mediante la formación en desarrollo web, APIs y sistemas distribuidos, que constituyen componentes esenciales de los sistemas de información modernos.

Asimismo, por integrar el trayecto del título intermedio de Programador/a de Sistemas, aporta a los alcances de dicho título vinculados con la definición de estrategias, técnicas y metodologías de programación; el desarrollo de sistemas que aseguren su funcionamiento según definiciones de performance establecidas; y el diseño e implementación de pruebas de funcionamiento, verificación y validación de software.

3.-EJES/COMPETENCIAS DE LA CARRERA A LOS QUE APORTA LA ASIGNATURA

EJE/COMPETENCIAS	APORTE		
	ALTO	MEDIO	BAJO
CE.AyL. 2: Interpretar especificaciones de modelos para el diseño e implementación de programas mediante la correcta aplicación de principios de los paradigmas de programación.		X	
CE.AyL. 4: Evaluar lenguajes de programación para determinar su grado de adecuación a un contexto de desarrollo específico analizando las características propias del paradigma y de la herramienta.		X	
CT 1. Identificación, formulación y resolución de problemas de informática.		X	
CT 2. Concepción, diseño y desarrollo de proyectos de informática.		X	
CT 4. Utilización de técnicas y herramientas de aplicación en la informática.		X	
CT 5. Generación de desarrollos tecnológicos e innovaciones tecnológicas.		X	
CSPA 6. Fundamentos para el desempeño en equipos de trabajo.		X	

EJE/COMPETENCIAS	APORTE		
	ALTO	MEDIO	BAJO
CSPA 7. Fundamentos para la comunicación efectiva.		X	
CSPA 8. Fundamentos para la acción ética y responsable.		X	
CSPA 10. Fundamentos para el aprendizaje continuo.		X	

4.- RESULTADOS DE APRENDIZAJE

Al finalizar la asignatura, de acuerdo a los contenidos teóricos y prácticos que se proponen el/la estudiante deberá alcanzar los siguiente Resultados de Aprendizaje (RA)

- RA 1: Desarrolla aplicaciones web integrales mediante la implementación de arquitecturas cliente-servidor para resolver problemas de intercambio de información siguiendo los estándares de los protocolos de comunicación actuales.
- RA 2: Crea interfaces de usuario interactivas y adaptativas utilizando lenguajes de marcado, estilos y programación asíncrona del lado del cliente, aplicando criterios de usabilidad y accesibilidad, para optimizar la experiencia de usuario según principios de diseño moderno.
- RA 3: Construye servicios web y APIs robustas mediante la programación del lado del servidor y la gestión de persistencia para garantizar la disponibilidad y seguridad de los datos bajo estándares REST, de autenticación y de seguridad, verificando su funcionamiento mediante pruebas automatizadas.
- RA 4: Optimiza el rendimiento de aplicaciones mediante la aplicación de modelos de concurrencia y algoritmos paralelos, evaluando la mejora obtenida con métricas de aceleración (speedup) y eficiencia, para maximizar la capacidad de cómputo y la respuesta del sistema en entornos multihilo.
- RA 5: Diseña sistemas distribuidos mediante el uso de mecanismos de comunicación entre nodos, WebSockets y algoritmos de coordinación para asegurar la interoperabilidad y la tolerancia a fallos en infraestructuras de red complejas.

5.-CONTENIDOS MÍNIMOS (PLAN DE ESTUDIOS)

Sistemas web. Aplicaciones cliente-servidor. Algoritmos distribuidos y paralelos. Programación distribuida y paralela. Elementos de diseño de interfaz de usuario.

6.-PROGRAMA ANALÍTICO

Unidad 1: Fundamentos de la Web y diseño de interfaces de usuario

Internet y World Wide Web. Modelo Cliente/Servidor en la Web. Componentes de la Web. Protocolo HTTP. Patrón de comunicación, concepto de mensajes, métodos. Estructuración de documentos HTML. Elementos básicos y semánticos. Estilos con CSS. Disposición de elementos. Frameworks CSS. Técnicas de Diseño Responsivo. Elementos de diseño de interfaz de usuario: principios de diseño visual (jerarquía, contraste, alineación y consistencia). Heurísticas de usabilidad. Patrones de interacción y de navegación. Accesibilidad web: pautas WCAG, HTML semántico y ARIA. Prototipado de baja y alta fidelidad.

Resultados de aprendizaje asociados: RA 1, RA 2.

Unidad 2: Programación del lado del cliente

JavaScript. Sintaxis y construcciones básicas. Funciones y Manejo de Eventos. Modelo de ejecución asincrónico: event loop, callbacks, promesas y async/await. Consumo de APIs desde el cliente: DOM y Fetch. Definición, procesamiento y validación de formularios y retroalimentación al usuario.

Resultados de aprendizaje asociados: RA 1, RA 2.

Unidad 3: Programación del lado del servidor

Características de la programación del lado del servidor web. Procesamiento de solicitudes HTTP. Entrega de recursos estáticos. Construcción de APIs REST. Buenas prácticas en el diseño de APIs REST. Seguridad y manejo de errores. Autenticación y Autorización. Persistencia de datos desde la aplicación: acceso a bases de datos mediante controladores (nivel introductorio). Seguridad en aplicaciones web: HTTPS, CORS y principales vulnerabilidades (OWASP Top 10: inyección, XSS, CSRF). Pruebas automatizadas de APIs.

Resultados de aprendizaje asociados: RA 1, RA 3.

Unidad 4: Concurrencia y programación paralela

Concurrencia y paralelismo: conceptos y diferencias. Modelos de concurrencia: hilos y memoria compartida, paso de mensajes, event loop y programación asincrónica, worker threads en Node.js. Sincronización en memoria compartida: condiciones de carrera, exclusión mutua, locks y semáforos. Algoritmos paralelos: divide y vencerás y fork-join. Medición del rendimiento: aceleración (speedup), eficiencia y ley de Amdahl.

Resultados de aprendizaje asociados: RA 4.

Unidad 5: Sistemas distribuidos

Fundamentos y desafíos de los sistemas distribuidos: ausencia de memoria compartida y de reloj global, fallas parciales. Comunicación entre nodos: sincrónica (REST), asincrónica (colas de mensajes) y en tiempo real (WebSockets). Tiempo y orden de eventos: relojes lógicos de Lamport. Coordinación: elección de líder (algoritmo Bully). Replicación y consistencia. Tolerancia a fallos y nociones de consenso. Particiones y disponibilidad: teorema CAP.

Resultados de aprendizaje asociados: RA 5.

7.-BIBLIOGRAFÍA

Obligatoria

- Connolly, R., & Hoar, R. (2022). *Fundamentals of web development* (3.ª ed.). Pearson.

- Flanagan, D. (2020). *JavaScript: The definitive guide* (7.ª ed.). O'Reilly Media.
- Haverbeke, M. (2024). *Eloquent JavaScript* (4.ª ed.). No Starch Press.
- Brown, E. (2019). *Web development with Node and Express: Leveraging the JavaScript stack*. O'Reilly Media.
- Coulouris, G., Dollimore, J., & Kindberg, T. (2012). *Distributed systems: Concepts and design* (5.ª ed.). Pearson.
- Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.
- Hunter, T., & English, B. (2021). *Multithreaded JavaScript: Concurrency beyond the event loop*. O'Reilly Media.
- van Steen, M., & Tanenbaum, A. S. (2023). *Distributed systems* (4.ª ed.). distributed-systems.net. Disponible en acceso abierto.
- Tidwell, J., Brewer, C., & Valencia, A. (2019). *Designing interfaces: Patterns for effective interaction design* (3.ª ed.). O'Reilly Media.

Complementaria

- Meyer, E. A. (2017). *CSS: The definitive guide* (4.ª ed.). O'Reilly Media.
- Marcotte, E. (2014). *Responsive web design* (2.ª ed.). A Book Apart.
- Masse, M. (2011). *REST API design rulebook*. O'Reilly Media.
- Grinberg, M. (2018). *Flask web development* (2.ª ed.). O'Reilly Media.
- Krug, S. (2014). *Don't make me think, revisited: A common sense approach to web usability* (3.ª ed.). New Riders.
- W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/
- Mozilla. (s.f.). *MDN Web Docs* [Documentación en español]. https://developer.mozilla.org/es/

8.- METODOLOGÍA DE ENSEÑANZA

La propuesta pedagógica se basa en un enfoque Basado en Competencias y Centrado en el Estudiante, promoviendo su participación activa y el desarrollo de competencias vinculadas al ejercicio profesional siempre en coherencia con los alcances del título y las actividades profesionales reservadas.

Los contenidos teóricos y prácticos se abordarán de manera articulada mediante estrategias didácticas orientadas a la resolución de problemas, actividades aplicadas y prácticas de laboratorio, haciendo uso de herramientas y tecnologías actualmente demandadas por la industria del desarrollo de software y destacando las experiencias profesionales de los profesores y estudiantes.

Como soporte tecnológico se utilizarán software de código abierto, los equipos de las salas de máquinas de nuestra Facultad de Ciencias de la Administración y el Campus Virtual UNER, que funcionará como espacio para el dictado de clases, la publicación de materiales, la realización de autoevaluaciones, la entrega de proyectos y la comunicación entre docentes y estudiantes.

Estrategias de enseñanza. Se desarrollarán clases teórico-prácticas con demostraciones de código en vivo y actividades prácticas breves en clase (HTML, CSS, JavaScript, Node.js y Express, y luego ejemplos de concurrencia y de algoritmos distribuidos), pensadas para que las dudas surjan y se resuelvan en el

momento. En paralelo, los estudiantes desarrollan en equipos un trabajo final integrador incremental, que avanza semana a semana acompañando la secuencia de las unidades: comienza como una aplicación web cliente-servidor y evoluciona hacia una versión concurrente y distribuida. El uso de control de versiones (Git) y la revisión de código entre pares se incorporan como prácticas habituales de trabajo.

Distribución de la carga horaria. De las 128 horas totales, aproximadamente 48 horas (37,5 %) se destinan a actividades teóricas y 80 horas (62,5 %) a formación práctica (laboratorio, resolución de problemas y desarrollo del proyecto integrador), lo que supera el mínimo del 50 % de formación práctica establecido por el plan de estudios.

Uso de herramientas de inteligencia artificial generativa. Los asistentes de programación basados en inteligencia artificial forman parte del ejercicio profesional actual, por lo que su uso se incorpora de manera explícita y guiada. Se habilitan en actividades determinadas como apoyo para explorar alternativas, depurar y documentar código, y se exige que los estudiantes declaren su uso y sean capaces de explicar, justificar y modificar todo el código que entregan. Las instancias de evaluación individual (parcial y defensa del trabajo final integrador) verifican esa comprensión. Se promueve, además, una mirada crítica sobre la calidad, la seguridad y las implicancias éticas del código generado.

Trabajos prácticos y trabajo final integrador

La práctica de la asignatura se organiza en dos niveles complementarios:

- **Actividades prácticas en clase.** Ejercicios breves de HTML, CSS, JavaScript, Node.js y Express (y, en la segunda parte de la cursada, ejemplos de concurrencia y de algoritmos distribuidos) que se resuelven durante la clase con acompañamiento docente. Su propósito es que cada concepto se aplique inmediatamente después de presentado y que las dudas surjan y se resuelvan en vivo.

- **Trabajo final integrador (TFI).** Un proyecto en equipo que los estudiantes desarrollan de manera incremental a lo largo de todo el cuatrimestre. Cada unidad incorpora una nueva capa al proyecto: la interfaz de usuario (Unidades 1 y 2), la API, la persistencia y la seguridad (Unidad 3), el procesamiento concurrente (Unidad 4) y los componentes distribuidos (Unidad 5). Se prevén una entrega de avance a mitad de la cursada y la entrega final, según el cronograma (punto 11). El problema del TFI se toma de casos reales: en el primer cuatrimestre de 2026, un sistema de inscripción a cursos para la Facultad; en el segundo, la implementación de un sistema de gestión de incidencias y tareas de reparación para la Dirección de Informática de la Municipalidad de Concordia, cuyo análisis y diseño se elaboró en Ingeniería de Software I. De este modo, los estudiantes recorren el camino completo desde la especificación hasta el sistema en funcionamiento.

- **Trabajos prácticos de uso de inteligencia artificial.** Dos trabajos prácticos breves, centrados en la comprensión y no en la producción de código: los estudiantes reciben código generado por un asistente de IA y deben explicar qué hace, detectar sus errores y corregirlos justificando cada decisión. El primero, en la Unidad 3, se enfoca en la seguridad de una API (validación de datos, autenticación, vulnerabilidades OWASP); el segundo, en la Unidad 4, en la detección de una condición de carrera. Se entregan con una breve explicación escrita y se discuten en clase.

Seguimiento y atención de las dificultades de aprendizaje

- **Evaluación diagnóstica inicial.** En la primera semana de clases, los estudiantes resuelven un ejercicio práctico breve sobre los fundamentos necesarios para la asignatura (HTTP, URL, HTML, JavaScript básico). No lleva calificación: permite al equipo docente ajustar el punto de partida, identificar tempranamente a quienes necesitan acompañamiento adicional y mostrar a cada estudiante la distancia entre lo que cree saber y lo que efectivamente puede resolver.

- **Exposición de avances del TFI en clase.** Los equipos presentan periódicamente el estado de su trabajo final integrador. Esta instancia permite detectar de forma temprana a los estudiantes y equipos con dificultades, compartir soluciones entre pares y orientar los próximos pasos de cada proyecto.
- **Clases de consulta.** El equipo docente ofrece clases de consulta, previa cita, para trabajar de manera personalizada las dificultades detectadas.
- **Consultas en el Campus Virtual.** El foro de consultas permite resolver dudas entre clases y deja las respuestas disponibles para todo el curso.

9. - METODOLOGÍA DE EVALUACIÓN

La evaluación se concibe como una instancia integral del proceso de enseñanza-aprendizaje, no limitada a la acreditación, sino orientada al aprendizaje continuo y a la mejora progresiva.

A lo largo de la asignatura se evaluará permanentemente el proceso de enseñanza-aprendizaje. En este sentido, la evaluación no se considera limitada a la sino que se entiende como un proceso integral donde la misma es formativa y de seguimiento, tanto en actividades teóricas como prácticas, enfocada en el desarrollo de competencias profesionales propias del área de la programación.

Se considerarán como criterios mínimos de evaluación los siguientes aspectos:

- Comprensión conceptual y aplicación del conocimiento: se valorará el dominio de los conceptos fundamentales vinculados al desarrollo web tanto del lado del cliente como del servidor, los sistemas distribuidos y la programación concurrente, así como la capacidad para aplicarlos en la resolución de problemas concretos y el uso adecuado del lenguaje técnico.
- Desarrollo de habilidades prácticas: se evaluará la correcta implementación de aplicaciones y servicios, el uso apropiado de herramientas y entornos de desarrollo, la resolución eficiente de consignas prácticas y la transferencia de los conceptos teóricos a soluciones funcionales y escalables.
- Trabajo colaborativo y participación activa: se considerará la participación en clase, el trabajo en equipo, la capacidad para integrar aportes en proyectos grupales, la responsabilidad en el cumplimiento de tareas y el respeto por buenas prácticas de desarrollo de software.
- Comunicación técnica y académica: se valorará la claridad y coherencia en la documentación, presentaciones y entregas, el uso adecuado del lenguaje técnico, así como el cumplimiento de plazos y consignas establecidas.

Instrumentos de evaluación. Se utilizarán rúbricas para los trabajos prácticos y el proyecto integrador, conocidas por los estudiantes desde el inicio de la cursada; autoevaluaciones en el Campus Virtual; un parcial individual de carácter teórico-práctico; y la defensa oral del trabajo final integrador.

10. -RÉGIMEN DE EVALUACIÓN Y ACREDITACIÓN

Acorde a lo establecido en el Reglamento Académico vigente Resolución “C.D.” 509/16, y modificatorias (t.o. por la Resolución de Decano Nº 310/21), a fines de la regularidad se considerarán:

- la entrega en tiempo y forma y aprobación de trabajos prácticos, actividades de laboratorio y autoevaluaciones de seguimiento.

- la aprobación de un (1) parcial o su correspondiente recuperatorio, de carácter individual y teórico-práctico. Se opta por una única instancia de parcial porque el seguimiento continuo se realiza mediante los trabajos prácticos y el proyecto integrador, que constituye el principal instrumento de integración de los contenidos.

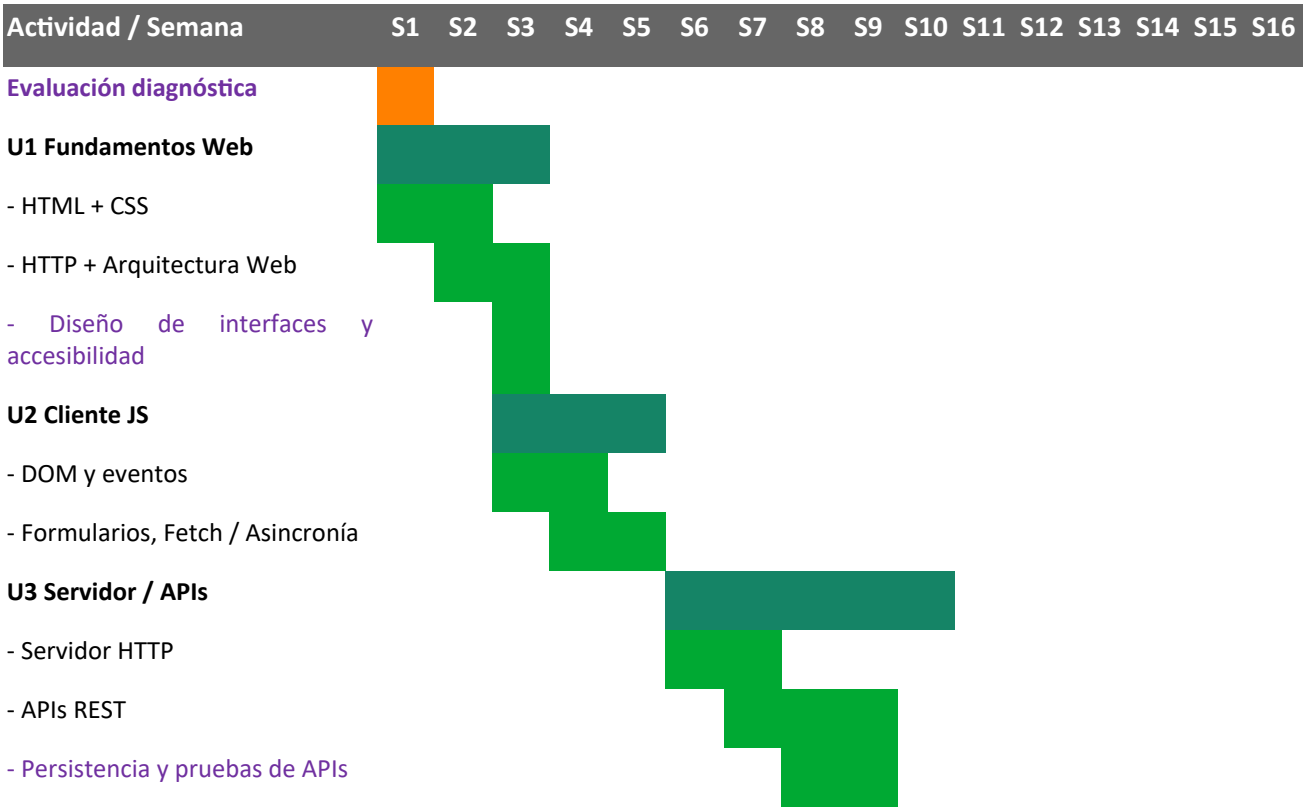
Para la acreditación de la asignatura se establece un régimen de promoción directa. En este marco, el/la estudiante deberá cumplir con los requisitos de regularidad mencionados previamente y, además, presentar y aprobar un trabajo final integrador. En caso de no aprobar dicho trabajo, el/la estudiante deberá rendir el examen final en los turnos establecidos por el calendario académico, a fin de alcanzar la acreditación de la asignatura. El trabajo final integrador se desarrolla en equipos de hasta cinco integrantes, consiste en la evolución del proyecto realizado durante el cursado e incorpora componentes concurrentes y distribuidos. Se aprueba mediante una defensa oral individual, en la que cada integrante debe dar cuenta del diseño y del código, con una calificación igual o superior a 6 (seis). Los estudiantes regulares que no alcancen la promoción rendirán un examen final que consiste en la presentación y defensa del trabajo final integrador y en preguntas conceptuales sobre los contenidos del programa.

Aquellos/as estudiantes que no cumplan con las condiciones de regularidad quedarán en condición de libre, pudiendo presentarse directamente a mesa de examen final en los turnos correspondientes. En ese caso, deberán aprobar previamente una instancia escrita de resolución de problemas prácticos y, luego, un examen oral sobre la totalidad de los contenidos del programa.

En todas las instancias de evaluación se aplicará la escala de calificaciones vigente en la universidad.

11. Cronograma de trabajo

A continuación se detallan mediante diagrama de Gantt las actividades de evaluación y los temas a desarrollar por semana.



- Auth / Seguridad (OWASP)

U4 Concurrencia

- Modelos de concurrencia y sincronización

- Algoritmos paralelos y medición de rendimiento

Trabajos prácticos de IA

Examen Parcial

U5 Distribuidos

- Fundamentos y comunicación entre nodos

- Relojes de Lamport y elección de líder

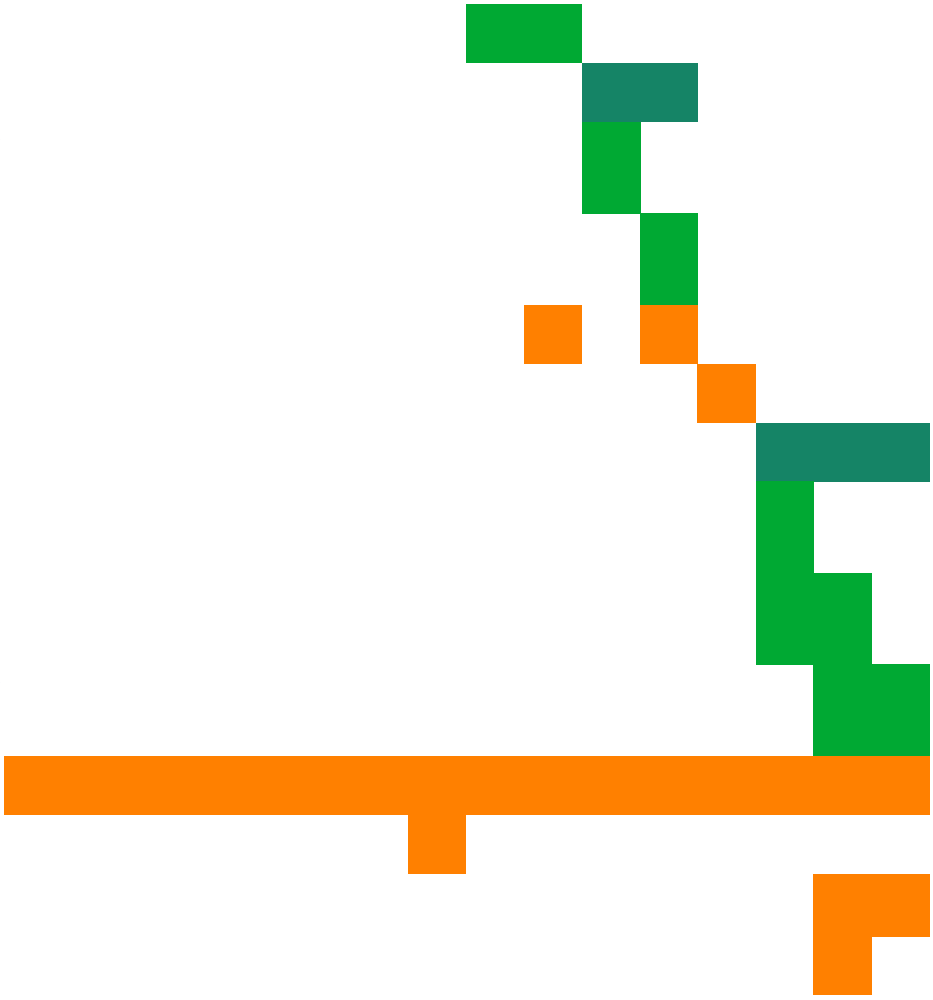
- Replicación, tolerancia a fallos y CAP

Proyecto integrador

- Entrega de Avance

- Entrega final

Examen Recuperatorio



12. Herramientas a utilizar

Para el desarrollo de las actividades teóricas y prácticas de la asignatura se emplearán herramientas y tecnologías actuales del ecosistema de desarrollo web y sistemas distribuidos, que permitan a los estudiantes experimentar el ciclo completo de construcción de aplicaciones modernas, desde la interfaz de usuario hasta el despliegue en entornos de red. En particular se utilizarán:

Herramientas pedagógicas:

- Pizarrón para la explicación de conceptos, esquemas de arquitecturas y resolución de problemas en clase.
- Presentaciones digitales elaboradas por el equipo docente para la introducción y sistematización de contenidos teóricos.
- Campus Virtual UNER como espacio de publicación de materiales, consignas, guías prácticas y comunicación con los estudiantes.
- Guías de trabajos prácticos y consignas de laboratorio orientadas al desarrollo incremental de aplicaciones.
- Ejemplos de código y demostraciones en vivo.
- Repositorios de código compartido con ejemplos, ejercicios y soluciones de referencia.

- Material bibliográfico digital y recursos web seleccionados por la cátedra.
- Foros o espacios de consulta virtual para acompañamiento del aprendizaje.
- Instancias de revisión colectiva de soluciones y análisis de casos en clase.

Herramientas de desarrollo:

- Lenguajes y estándares web (HTML5, CSS3 y JavaScript) para la construcción de interfaces y aplicaciones del lado del cliente.
- Navegadores web.
- Entornos de ejecución y desarrollo para JavaScript, Java y Python del lado del servidor. Por ejemplo: NodeJS 22 o superior, Python 3.12 o superior, JDK 21 o superior.
- Frameworks del lado del servidor para la construcción de servicios web y APIs REST. Ejemplo: Express, FastAPI o similar.
- Sistemas gestores de bases de datos relacionales para la persistencia de información. Por ejemplo: PostgreSQL, MySQL.
- Tecnologías de comunicación en tiempo real. Por ejemplo: WebSockets.
- Herramientas de control de versiones y repositorios de código compartido (Git, GitHub, GitLab).
- Entornos de desarrollo integrados (IDE) o editores de código para la implementación de aplicaciones. Por ejemplo: Visual Studio Code, Windsurf, NetBeans.
- Herramientas de prueba, depuración y validación de aplicaciones web. Por ejemplo: Herramientas del Desarrollador Postman, Bruno y frameworks de pruebas JavaScript. Herramientas de evaluación de accesibilidad web. Por ejemplo: Lighthouse, axe.
- Plataformas de virtualización o contenedores. Por ejemplo: Docker, DockerHub.
- Campus Virtual UNER como soporte para materiales, prácticas y entregas.

13.- Propuesta de formación de recursos humanos

La formación de recursos humanos constituye un eje estratégico para asegurar la sostenibilidad y el impacto de la asignatura. En este sentido, se proyecta la incorporación progresiva de docentes auxiliares, lo que permitirá consolidar la formación académica de nuevas generaciones de profesionales de la enseñanza. Paralelamente, se promoverá que los integrantes del equipo docente profundicen su participación en proyectos de investigación/extensión y, en la medida de lo posible, impulsen iniciativas en el marco del programa de proyectos de investigación y desarrollo PID NOVEL de la UNER. Estas estrategias buscan fortalecer la capacidad institucional de innovación y transferencia social, potenciando la articulación entre docencia, investigación y extensión.

14.- OBSERVACIONES

Cualquier otra cuestión que se considere necesaria